INF384 · TEMAS AVANZADOS EN INGENIERÍA DE SOFTWARE 1 · 2026-2

Sesión 04

Integración continua: validación de seguridad

Qué controles de seguridad entran al pipeline de integración, bajo qué condición detienen y cómo se diseña su umbral

Capítulo 3 (2 de 2) · 3 horas | RA2 · RA3 | Prof. Jose del Castillo Salazar

PUCP

INF384 · 9 CAPÍTULOS · 14 SESIONES · 42 HORAS

La ruta del curso



SESIÓN 4

Objetivo de la sesión y ruta

OBJETIVO

Diseñar la política de validación de seguridad de un pipeline de integración continua y justificar qué detiene, qué registra y qué queda fuera de esta etapa.

1

Revisión Lab 2

2

Qué entra a la integración

Criterios para que un control de seguridad viva en esta etapa, y el que no cumple ninguno.

3

Diseño del gate

Bloquear o registrar, umbral por severidad y tipo, y la excepción con fecha de vencimiento.

4

Caso Grupal

Resultados de aprendizaje vinculados: RA2 y RA3.

CONTINUIDAD CON LA SESIÓN 3 Y EL LABORATORIO 2

Dónde estamos en el pipeline

Commit	Build	Pruebas	Calidad	Seguridad

Lo que ya está resuelto

- Las dependencias se instalan desde el archivo de bloqueo: el mismo commit produce el mismo artefacto.
- Las pruebas producen un reporte de cobertura que el análisis de calidad consume.
- El umbral de calidad está declarado en el repositorio y detiene el pipeline cuando no se cumple.
- El artefacto se publica versionado, desde la rama principal y solo si la validación pasó.

Lo que ese pipeline no responde

- ¿Alguno de los componentes que instaló tiene una vulnerabilidad conocida?
- ¿Hay una credencial escrita en el repositorio o en su historial?
- ¿El código introduce un patrón que alguien puede explotar?
- ¿Qué componentes exactos viajan dentro del artefacto que se publicó?

A

Laboratorio 2

Los cuatro defectos y lo que generaba cada uno

1

Instalación sin respetar el archivo de bloqueo

Se pierde la reproducibilidad. El mismo commit puede producir dos artefactos distintos en dos días distintos.

2

Sin caché de dependencias

Se pierde tiempo de respuesta. El costo completo de resolución e instalación se paga en cada ejecución.

3

Análisis de calidad sin condición de detención

Se pierde el control. El análisis produce un reporte, y un reporte no detiene nada.

4

Publicación sin versión, sin rama y sin condición

Se pierde la trazabilidad. No hay forma de responder qué artefacto exacto salió, desde dónde y con qué validaciones.

Ninguno de los cuatro hacía fallar el pipeline. Todas las ejecuciones terminaban en verde.

El defecto que explica la duración

De las cuatro correcciones, la caché es la única cuyo efecto se observa directamente en la duración del pipeline. Las otras tres cambian garantías, no números de tiempo.

Lo que midieron

- Tres ejecuciones de línea base antes de intervenir
- La diferencia se concentra en el paso de instalación de dependencias; el resto de los pasos no cambia.
- La primera ejecución después de habilitar la caché no mejora: todavía no hay nada guardado.

Las dos métricas alcanzables sin desplegar

- Tiempo de entrega de cambios, medido de forma parcial: del commit al artefacto publicado.
- Tasa de fallo de cambios, medida como proporción de ejecuciones que rompen la rama principal.
- Frecuencia de despliegue y tiempo de restauración exigen despliegues reales.

La medición previa es lo que convierte la intervención en evidencia. Sin línea base, el número final no prueba nada

LABORATORIO 2

El punto 4.3 del entregable: lo que no se resolvió

1

Un componente con vulnerabilidad conocida

El pipeline instala el árbol completo desde el archivo de bloqueo y no compara ninguna versión contra nada.

2

Una credencial en el repositorio

Nada revisa el código ni el historial buscando tokens, llaves o contraseñas escritas.

3

Un patrón de código explotable

El análisis de calidad clasifica algunos hallazgos como riesgo, pero no es su foco.

4

El contenido exacto del artefacto

Se publica con versión y referencia del commit, pero sin inventario de los componentes que viajan dentro.

Las cuatro limitaciones comparten una causa: el pipeline valida cómo está escrito el código y no valida qué contiene el artefacto ni de dónde viene.
DIferencia entre construcción (BUILD) y una pregunta de riesgo (SEC)

B

Qué entra a la integración continua

Criterios para que un control de seguridad viva en esta etapa, los cuatro que los cumplen y el que no

POR QUÉ EL CONTROL ENTRA TEMPRANO

El mismo hallazgo cuesta distinto según dónde aparece

En el commit	Antes de fusionar	Antes de publicar	En producción
Lo corrige quien escribió el código, con el contexto todavía fresco y antes de que exista un artefacto.	Lo corrige el autor, pero ya hay commit de por medio y el cambio compite con el trabajo del resto del equipo.	Entra un equipo distinto, se reabre el ciclo de validación completo y la fecha comprometida se mueve.	Hay incidente y hay exposición. Además de corregir, hay que reconstruir qué versión estuvo expuesta y por cuánto tiempo.

El control tardío requiere más personas involucradas y más retrabajo.
En los **VSM** en **¿qué se traduce esto?**

Adelantar el control es una decisión de costo.

Cuándo un control pertenece a esta etapa

1. **Rápido**
   Se ejecuta en el orden de minutos. Un control que duplica la duración del pipeline termina moviéndose a una ejecución nocturna, y ahí deja de detener nada.

2. **Determinista**
   La misma entrada produce el mismo resultado. Un control que alterna verde y rojo sin que el código cambie enseña al equipo a reintentar hasta que pase.

3. **Acotable al cambio**
   Puede limitar el hallazgo a lo que este commit introdujo. Sin eso, un proyecto con historia queda bloqueado desde la primera ejecución.

4. **No exige ambiente desplegado**
   Se ejecuta sobre el código fuente y sus dependencias. Si necesita la aplicación corriendo y respondiendo, pertenece a una etapa posterior.

Son los mismos cuatro criterios que ya aplicaron al umbral de calidad, con otro tipo de hallazgo.

Controles que califican y no califican

Detección de secretos	Análisis de composición	Análisis estático de seguridad	Análisis de configuración e IaC
Busca credenciales, tokens y llaves en el código y en el historial del repositorio.	Compara el árbol de dependencias declaradas y transitivas contra bases de vulnerabilidades conocidas.	Busca patrones de código explotables recorriendo el código fuente, sin ejecutarlo.	Evalúa plantillas de infraestructura y archivos de configuración contra reglas escritas como código.
Cumple los cuatro criterios	*Cumple los cuatro criterios*	*Cumple los cuatro criterios*	*Cumple los cuatro criterios*

ANÁLISIS DINÁMICO DE SEGURIDAD · NO PERTENECE A ESTA ETAPA

Requiere la aplicación desplegada y respondiendo peticiones. Además suele necesitar varios minutos en ejecutarse..

Secretos en el repositorio

Es el control más barato de instalar y el que produce hallazgos menos discutibles.

1

Quitar la línea no resuelve nada

El valor sigue en el historial de git y hay que asumir que ya fue leído. La única remediación real es rotar la credencial.

2

Son dos controles, no uno

Uno se ejecuta sobre el cambio e impide que entre un secreto nuevo. Otro recorre el historial y encuentra los que ya entraron.

3

El hallazgo tiene dueño inmediato

A diferencia de una vulnerabilidad heredada, aquí siempre hay alguien del equipo que puede corregirlo hoy.

4

La causa raíz suele ser necesidad/desconocimiento y no descuido

Si el proyecto no ofrece una forma cómoda de inyectar configuración por ambiente, alguien va a escribir la credencial en el código.

Del árbol de dependencias al hallazgo

El mismo árbol de dependencias declaradas se valida componente por componente contra bases públicas de vulnerabilidades.

1. **El archivo de bloqueo es el requisito**
   Sin versión exacta no hay hallazgo exacto. Sobre un rango de versiones el control no puede afirmar ni descartar nada.

2. **El volumen cae en lo transitivo**
   La mayoría de hallazgos aparece en componentes que nadie del equipo eligió. Es lo que vuelve difícil la remediación.

3. **No se corrige lo vulnerable, se corrige lo que lo genera**
   Hay que subir la versión de la dependencia declarada que trae el componente afectado, y eso puede no existir todavía.

4. **Produce además el inventario del artefacto, las vulnerabilidades son vivas, pueden salir en cualquier momento.**
   La lista de componentes que viajan dentro. Es lo que responde «¿estoy afectado?» el día que se publica una vulnerabilidad crítica.

Todo proyecto con dependencias tiene componentes vulnerables. La pregunta operativa es cuánto tardan en responder si están afectados.

Severidad no es prioridad

1

Severidad declarada

La puntuación estándar describe el impacto potencial del defecto en abstracto. No conoce su sistema ni su arquitectura.

2

Alcanzabilidad

Si la función vulnerable nunca se ejecuta desde la aplicación, el riesgo real cambia. Determinarlo cuesta análisis y no siempre es concluyente.

3

Exposición

No es lo mismo un componente dentro de un servicio expuesto a internet que uno que corre en red local.

4

Existencia de corrección

Un hallazgo crítico sin versión corregida disponible exige una decisión distinta a uno que se resuelve subiendo un número.

Ordenar por severidad es automático; priorizar exige contexto que la herramienta no tiene.

Por qué son dos compuertas y no una

	Análisis de calidad	Análisis de seguridad
Qué pregunta	¿Está bien construido?	¿Es explotable?
Sobre qué evalúa	El código que este cambio agregó	El cambio y el árbol completo de dependencias
Costo del falso positivo	Ruido: el equipo aprende a ignorar	Ruido + ignorar hallazgo verdadero
Costo del falso negativo	Deuda técnica acumulada	Incidente con exposición
Quién autoriza la excepción	El equipo de desarrollo	Ideal: El equipo junto a quien responde por el riesgo Normalmente: Equipo Seguridad

C

Diseño del gate de seguridad

Qué detiene, qué solo registra, y qué pasa con lo que no se va a corregir hoy

Bloquear o registrar

Bloquear todo desde el primer día

En un proyecto con historia, el primer análisis devuelve cientos de hallazgos preexistentes y el pipeline queda rojo de forma permanente.
El equipo desactiva el control o aprende a saltarlo.

Registrar siempre, detener nunca

El número queda en un reporte que nadie abre y empeora cada mes sin que exista un momento en que detenerlo sea obligatorio. Simil al análisis de calidad sin umbral.

1 Detiene lo nuevo, inventaría lo heredado

El cambio de hoy no puede introducir un hallazgo por encima del umbral. Lo preexistente se registra y se planifica, no se ignora.

2 Umbral por tipo de control, no uno solo

Un secreto detectado detiene siempre. Una dependencia con severidad media y parche disponible puede registrarse y planificarse.

3 La condición se escribe antes del hallazgo

Igual que el umbral de cobertura: la discusión sobre qué es inaceptable se tiene en frío y queda versionada en el repositorio.

La excepción con fecha de vencimiento

1. **Qué hallazgo, con identificador**
   Una exclusión que apaga una regla completa para todos los posibles hallazgos no es una excepción, es desactivar el control.

2. **Escribir la razón por lo que no se corrige ahora**
   No hay versión corregida, el componente no es alcanzable, el cambio requiere una migración mayor.

3. **Quién la autoriza**
   Una excepción sin responsable identificable es una decisión que nadie tomó y por la que nadie responde.

4. **Hasta cuándo**
   Al vencer, el pipeline vuelve a detenerse y la decisión se toma de nuevo con la información de ese momento.

El número de excepciones vigentes es la métrica que importa: si crece mes a mes, el umbral está mal calibrado o falta capacidad de remediación.

Anatomía de la validación de seguridad

GITHUB ACTIONS — TRABAJOS DE SEGURIDAD

```
jobs:
  secretos:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }       # 1
      - run: <accion-de-deteccion>
  dependencias:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: <accion-de-composicion>   # 2
  estatico:
    runs-on: ubuntu-latest
    permissions:
      security-events: write           # 3
    steps:
      - run: <accion-de-analisis>
  publicar:
    needs: [calidad, secretos,
            dependencias, estatico]    # 4
```

1 Historial completo

La detección de secretos revisa lo que ya entró, no solo el cambio de hoy.

2 Se ejecuta sobre el archivo de bloqueo

Sin versión exacta de cada componente, el hallazgo no es exacto.

3 Permiso acotado

El pipeline declara solo lo que necesita escribir. El pipeline también es superficie de ataque.

4 Aquí está el gate

La publicación depende de los cuatro trabajos. Al no encadenarse entre sí, se ejecutan en paralelo.

Dónde cae cada control en el marco de referencia

1

PW.4 · Reutilizar software existente de forma segura

Análisis de composición, control de lo que entra a la organización e inventario de componentes del artefacto.

2

PW.7 · Revisar y analizar el código legible

Análisis estático de seguridad y detección de secretos, junto con la revisión humana del cambio.

3

PW.8 · Probar el código ejecutable

Pruebas automatizadas y, cuando existe un ambiente donde ejecutarlo, análisis dinámico.

4

RV.1 · Identificar y confirmar vulnerabilidades de forma continua

El proceso que responde qué artefactos están afectados cuando aparece una vulnerabilidad nueva.

El marco describe prácticas y pide evidencia de que se ejecutan; no nombra herramientas.
El pipeline es la forma más barata de producir esa evidencia sin trabajo manual adicional.

ACTIVIDAD A1 · GRUPOS DE TRABAJO

Política de validación de seguridad

22 min

Consigna

- Sobre el caso transversal de su grupo, definan para cada uno de los cuatro controles si detiene el pipeline o solo registra el hallazgo.
- Justifiquen dos de esas cuatro decisiones con un dato del value stream map que levantaron en la Sesión 1.
- Definan la regla de excepción: quién autoriza y cuál es el vencimiento máximo que aceptan.
- Nombren el control que no van a activar todavía y la capacidad concreta que le falta al equipo del caso para sostenerlo.

CRITERIO DE EVALUACIÓN

Se evalúa la consistencia entre la restricción del caso y la política propuesta. No se evalúa qué tan exigente es el umbral elegido.

ERRORES FRECUENTES

- Que los cuatro controles detengan.
- Un mismo umbral para los cuatro.
- Excepciones sin responsable ni fecha.
- Activar el análisis dinámico en esta etapa.

Entregable: se sube al foro del aula virtual al terminar la sesión.

Kahoot de cierre

6 preguntas · 30 segundos cada una

PUNTOS ADICIONALES

- 1.er puesto: +1 punto en el examen parcial
- 2.º puesto: +0,5 en el examen parcial
- 3.er puesto: +0,5 en el examen parcial

CIERRE SESIÓN 4

Qué se llevan

1. **Un pipeline en verde no es un pipeline correcto**
   Los cuatro defectos del Laboratorio 2 pasaban la validación sin detener nada.

2. **Calidad y seguridad son dos preguntas distintas**
   Un solo umbral no puede responder por dos consecuencias que no se parecen.

3. **El control entra donde es rápido, determinista y acotable**
   Lo que exige un ambiente desplegado no pertenece a la integración continua.

4. **La severidad ordena, pero no prioriza**
   Sin alcanzabilidad, exposición y disponibilidad de corrección, el orden es arbitrario.

5. **La excepción sin fecha es una decisión permanente**
   Y una que nadie tomó explícitamente es la que reaparece en el postmortem.

PARA LA SESIÓN 5

- Capítulo 4, despliegue continuo: gestión de artefactos, infraestructura como código, observabilidad y flujo de aprobaciones.
- Leer NIST SP 800-218, prácticas PW.4 y RV.1. Revisar el punto 4.3 de su entregable: la limitación que nombraron abre la próxima sesión.